

VNUHCM – UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY

Linux Operating System & Applications

CAPSTONE PROJECT

Requirements, Instructions & Grading Rubric

Module	Module 5 — Scripting & Capstone Project
Sessions	Session 11 (build + live demo + defence)
Weight	20% of the final course grade
Base scale	10 marks + 1 bonus mark (self-signed TLS)
Environment	VirtualBox or VMware · Ubuntu 22.04 Server or CentOS Stream 9
Mode	Team of 5 students (see §2)
Assessment	Live demo + technical report + oral defence

No cloud accounts required. All work runs on local virtual machines.

1. Project Overview

The capstone is the culminating assessment of the course. It asks you to combine everything from Modules 1–5 — the command line, system administration, storage, networking, hardening, service deployment, and Bash automation — into one coherent, working Linux system that you build, secure, automate, document, and defend.

This is not a written exam. You are graded on a system that actually runs. During the demo you will boot your VM(s), show the services live, break something on purpose, and prove your automation and backups recover it. You then answer technical questions about the choices you made.

1.1 Learning outcomes assessed

On completing the capstone you will have demonstrated the ability to:

- Provision and configure an Ubuntu or CentOS server system from a clean install.
- Deploy and connect real services (web server, database, and/or an application) so they work together end to end.
- Harden the system to a defensible baseline: firewall, SSH policy, brute-force protection, and audit logging.
- Design and implement a working backup-and-restore process, and prove recovery live.
- Automate operational tasks in Bash — monitoring, backup, alerting, deployment — with proper error handling.
- Document the system to a professional standard and defend your engineering decisions under questioning.

1.2 How to read this document

Section 2 covers logistics. Section 3 is the mandatory common baseline. Sections 4–6 describe a single unified project made of three mandatory components: (4) Infrastructure, Web & Security, (5) Application & Database, (6) Operations & Automation — the team builds all three, there is no choice. Section 7 explains the optional TLS bonus. Sections 8–10 define the three deliverables (report, demo, defence). Section 11 is the full grading rubric. Section 12 is a requirements compliance matrix to tick off while you build, and Section 13 is the final pre-demo checklist.

2. Logistics

2.1 Environment

Hypervisor: Oracle VirtualBox or VMware Workstation/Player (choose one; record it in the report).

Guest OS: Ubuntu 22.04 LTS Server OR CentOS Stream 9 (no desktop environment). Choose one and use it consistently.

VM sizing: Minimum 2 vCPU, 2 GB RAM, 20 GB disk per VM. Keep it lean — the labs are designed for low-spec classroom machines.

Networking: A Host-Only or Internal network is sufficient (on VMware the equivalents are Host-only / LAN Segment). NAT is allowed for package installation. Record your chosen adapter mode in the report.

Number of VMs: At least 2 VMs are mandatory for a team of 5 — one main service VM and one used for backup replication or remote access (e.g. as an SSH ProxyJump bastion).

Distro tooling: Requirements are distro-neutral. Use the toolchain that matches your OS: apt / UFW / /etc/ssl on Ubuntu, or dnf / firewalld / /etc/pki/tls on CentOS. State your choice in the report; both are graded equally.

2.2 Team size & role division

Each team has 5 students. Because this is a large team, the expected operational depth is high: a multi-VM system, a rich automation suite, and security and backup fully implemented. Specific requirements for a team of 5:

- A 2-VM (or more) architecture is mandatory (see §2.1).
- A work-allocation table in the report, mapping the 5 members onto the three project components (§4–6). Suggested five roles: (1) infrastructure & networking, (2) reverse proxy & web security, (3) application & systemd, (4) database & backup, (5) automation toolkit & alerting.
- Marks are individual on top of a team mark: each member must defend their own part AND understand the system as a whole. A member who cannot explain their part loses individual marks even if the system runs well.
- During the defence the lecturer may ask any member about any component of the system.

Division of labour is not "everyone for themselves"

The whole team owns the whole system. Splitting the work only clarifies responsibility — every member must still understand how the components fit together. A disjointed system whose members cannot explain each other's parts scores poorly on the Report & defence criterion.

2.3 Timeline

When	What
Session 10	Confirm your team of 5 and begin building — you already have the Bash scripting foundation from this session.
Before Session 11	Complete the build. Write the technical report. Prepare a snapshot of a clean, working state so a demo failure is recoverable.
Session 11 (in class)	Live demo (≈12–15 min) + oral defence (≈8–10 min) per team.
Submission deadline	Upload report + all scripts/configs as a single archive by the end of Session 11 (exact time announced in class).

2.4 What to submit

One archive named

capstone_<groupID>.tar.gz	# e.g. capstone_G05.tar.gz
---------------------------	----------------------------

containing:

- The technical report (PDF).
- All Bash scripts you wrote, with the shebang and comments intact.
- All service configuration files you changed (Nginx/Apache, systemd units, database config, UFW/fail2ban/auditd rules) — real files, not screenshots.
- A short README.md stating the project components, VM layout, the 5-member work-allocation table, and exactly how to reproduce the setup.

Do not submit secrets

Never put real passwords, private keys, or API tokens in the archive. Replace them with placeholders (e.g. <DB_PASSWORD>) and explain in the report where each secret is stored on the running system. Committing a live secret is an automatic deduction under the security criterion.

3. Mandatory Common Baseline

Your system must satisfy all of the following. This is the non-negotiable core, mapping directly onto the Security, Backup, and Automation grading criteria, and it underpins the three components in Sections 4–6.

3.1 System & storage

- Clean Ubuntu 22.04 Server or CentOS Stream 9 install, hostname set to something meaningful (not the default).
- A non-root administrative user with sudo; day-to-day work is done as that user, not root.
- At least one dedicated data or backup location mounted via `/etc/fstab` and surviving a reboot. LVM is encouraged (you practised online expansion in Module 2) but a separate partition or virtual disk is acceptable.

3.2 Security hardening (maps to the 20% Security criterion)

- Host firewall enabled with a default-deny inbound policy — UFW on Ubuntu or `firewalld` on CentOS. Only the ports your services actually need are open. Justify every open port in the report.
- SSH hardened in `sshd_config`: root login disabled, key-based authentication configured, and at least one of {non-default port, `AllowUsers` restriction}. Password login may remain enabled for the demo but the report must note the trade-off.
- `fail2ban` installed and protecting SSH. You must be able to trigger a ban live (or show a recent ban in the jail status) during the demo.
- `auditd` (or equivalent) watching at least the sensitive files `/etc/passwd` and `/etc/shadow`, with a readable audit trail.
- A Lynis run recorded in the report with the hardening score and at least three remediated findings (as practised in Session 6).

3.3 Backup & restore (maps to the 20% Backup criterion)

- An automated backup of the data that matters for the system (database dump and/or application files and/or web content), compressed and timestamped.
- A retention policy that removes backups older than a stated age (e.g. 7 days), implemented in your script — not done by hand.
- The backup job scheduled via cron or a systemd timer, with the schedule documented.
- A restore procedure you can run live. During the demo you will destroy or corrupt the live data and restore it from backup, then prove the data is intact.

The restore is the point

A backup you have never restored is not a backup. The 20% for this criterion is earned by a live restore that visibly recovers lost data — not by showing that a backup file exists. Rehearse it.

3.4 Automation (maps to the 15% Automation criterion)

- At least one health-check script that inspects real system state (service up/down, port listening, disk/CPU/RAM thresholds) and reacts — logs and/or alerts — when something is wrong.
- Scripts use `set -euo pipefail` and a trap on `ERR/EXIT` so failures are caught rather than silently ignored (Session 10 material).
- Every script passes `shellcheck` with no errors. Include the clean `shellcheck` output or note remaining justified warnings in the report.
- Alerting is implemented by at least one channel: the mail command, `msmtp` email, or the Telegram Bot API via `curl`. A simulated failure must actually fire an alert during the demo.

4. Unified Project — Architecture & Component 1: Infrastructure, Web & Security

The whole project is ONE system

A team of 5 builds a single unified system made of three mandatory components: (1) Infrastructure, Web & Security — Section 4; (2) Application & Database — Section 5; (3) Operations & Automation — Section 6. There is no choice: all three must be built and must work together.

4.1 Overall architecture

The request flow through the system: browser/curl → Nginx/Apache (reverse proxy, ≥ 2 virtual hosts) → the application running under systemd (Section 5) → a relational database listening on localhost only. All operations — deployment, backup, monitoring — go through the Bash toolkit (Section 6). The system spans at least 2 VMs: one main service VM and one that receives the backup replica.

The three components map directly onto the grading criteria: Component 1 is where the Security criterion converges, Component 2 is the core "System works", and Component 3 carries Backup and Automation.

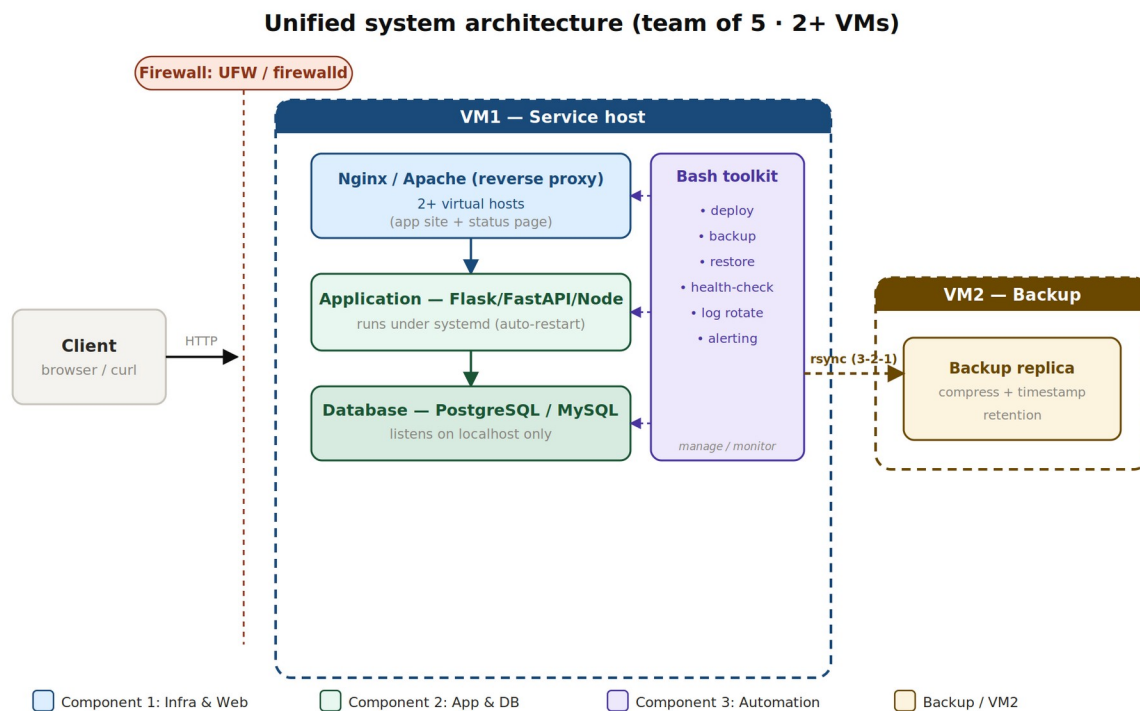


Figure 1 — Architecture of the unified system and its three mandatory components.

4.2 Requirements — Infrastructure, Web & Security

- Nginx or Apache as a reverse proxy in front of the Section 5 application (Nginx proxy_pass, or Apache mod_proxy/ProxyPass, to the app's local port), forwarding the correct headers.
- Besides the application site, serve at least one more virtual host (e.g. a static status page or a second site) → at least 2 virtual hosts on the same web server (Nginx or Apache).
- Full system hardening per §3.2 (firewall = UFW or firewalld to match your distro) — this is where the Security criterion converges.
- fail2ban is encouraged to also cover the web tier if the app exposes a login-like endpoint.

- A 2-VM (or more) architecture with a connection diagram in the report; if the second VM is a bastion, configure SSH ProxyJump (Session 5 material).

4.3 Demo must show (this component)

- Both virtual hosts responding, e.g. via `curl -H "Host: ..."` from the host or a second VM.
- A live fail2ban ban after simulated SSH brute-force.
- auditd catching access to `/etc/passwd` or `/etc/shadow`.

5. Component 2: Application & Database

Summary

A real Python (Flask/FastAPI) or Node.js application backed by a relational database (PostgreSQL or MySQL), run as a systemd service and proxied by the Section 4 web server (Nginx or Apache). This is the core "System works".

5.1 Requirements — Application & Database

- A working application (Flask, FastAPI, or Node.js/Express). It can be small, but it must talk to the database and expose at least one endpoint that reads and one that writes.
- A relational database — PostgreSQL or MySQL: a dedicated database and an application user/role with only the privileges it needs (least privilege), never using the DB admin account for the app.
- The database listens on localhost only (bind-address for MySQL, or `listen_addresses` / `pg_hba` for PostgreSQL) — not reachable from outside the host. Prove this with `ss` during the demo.
- The application run as a systemd service: a `.service` unit with `Restart=on-failure`, an `EnvironmentFile` for secrets (DB URL/credentials), and logs going to `journald` via `stdout/stderr`.
- Secrets kept out of the codebase — in the `EnvironmentFile` with restricted permissions — not hard-coded. The report must show the permission bits on that file.
- Database backup (`mysqldump` or `pg_dump`) is handled by the toolkit in Section 6, per the common baseline (§3.3).

5.2 Demo must show (this component)

- The app serving end to end through the web server (browser or `curl` → Nginx/Apache → app → database → response).
- `systemctl` status showing the app as an active, enabled service.
- A simulated crash (kill the app process) and systemd automatically restarting it, with the event visible in `journalctl`.
- `ss` output proving the database listens only on 127.0.0.1.

6. Component 3: Operations & Automation

Summary

A menu-driven Bash toolkit that controls the whole operational lifecycle of the system above: deploy, backup & restore, health-check, and log rotation. This is where the Backup and Automation criteria converge.

6.1 Requirements — Bash toolkit

- An interactive CLI menu (the entry point) that dispatches to each tool and handles invalid input gracefully.
- A deploy action: put the app/site into place, set ownership and permissions correctly, and (re)start the service safely (test → reload → confirm, with rollback if the reload fails).
- A backup action: dump the database (Section 5) + web content (Section 4), compress, timestamp, apply retention, and rsync to the second VM (the 3-2-1 idea from Session 10).
- A restore action: recover the database/content from a backup the toolkit itself produced.
- A health-check action: CPU, disk, RAM, listening ports, and service status (web server, app, DB) against configurable thresholds, with alerting on breach.
- A log-rotation action: rotate and compress a chosen log, keep N generations, remove older ones.
- Every tool: getopt-style parameters where sensible, set -euo pipefail, trap on ERR, clear logging, and a --help output. The whole toolkit passes shellcheck cleanly.

6.2 Demo must show (this component)

- The menu running and dispatching to each tool.
- A deploy that includes a deliberately broken config, the safe-reload catching it, and an automatic rollback to the last good state.
- A backup run producing a timestamped archive and rsyncing it to the second VM, plus the retention step removing an old one.
- Destroying live data then restoring it from a toolkit-produced backup, proving integrity.
- A health-check breach (e.g. stop a service) triggering an alert.
- shellcheck run live over the toolkit with a clean result.

7. TLS / HTTPS — Optional Bonus (+1 Mark)

SSL/TLS is not required for the base grade. The system is graded on HTTP. However, deploying a working self-signed TLS certificate correctly is worth one bonus mark on top of the 10-mark scale.

7.1 Why self-signed, and why not Let's Encrypt here

The Session 7 material teaches certbot / Let's Encrypt, but that path does not work on an isolated lab VM, and understanding why is part of the point:

- Let's Encrypt issues certificates only for a public domain name it can validate over the internet (the ACME challenge reaches your server on port 80/443 from outside).
- A VirtualBox VM on a Host-Only or Internal network has no public domain and is not reachable from the ACME servers, so domain validation fails.
- Therefore the correct local approach is a self-signed certificate (or a small internal CA you create yourself). Your report must state this reasoning to earn the bonus — running the command is not enough.

7.2 What earns the bonus mark

- Generate a self-signed certificate and key with openssl (or an internal CA + server cert for the stronger version).
- Configure Nginx (or Apache) to serve your site/app over HTTPS using it.
- Redirect plain HTTP to HTTPS.
- Open 443 in the firewall (UFW or firewallld) and confirm 80→443 behaviour.
- Demonstrate the HTTPS endpoint live (a browser trust warning is expected and fine for self-signed; curl -k or importing your CA both acceptable).
- Explain the §7.1 reasoning in the report.

7.3 Starting point

A minimal self-signed certificate (adapt paths and subject to your setup):

```
# Generate a 1-year self-signed cert + key
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout /etc/ssl/private/lab-selfsigned.key \
  -out /etc/ssl/certs/lab-selfsigned.crt \
  -subj "/C=VN/ST=HCMC/O=HCMUS-Lab/CN=lab.local"

# Nginx server block (essentials)
# listen 443 ssl;
# ssl_certificate /etc/ssl/certs/lab-selfsigned.crt;
# ssl_certificate_key /etc/ssl/private/lab-selfsigned.key;
# Plus an HTTP server block that returns 301 -> https://$host$request_uri;

# Apache: SSLEngine on; SSLCertificateFile + SSLCertificateKeyFile;
# redirect HTTP->HTTPS with Redirect permanent / or mod_rewrite.

# CentOS Stream 9: use /etc/pki/tls/certs and /etc/pki/tls/private instead,
# and open 443 with: sudo firewall-cmd --add-service=https --permanent && sudo
firewall-cmd --reload
```

The bonus is capped at +1 mark and cannot raise the total above the maximum allowed by your program's grading rules.

8. Deliverable 1 — Technical Report

A concise engineering report, submitted as PDF, roughly 6–12 pages excluding appendices. Quality and correctness matter far more than length. Suggested structure:

- **System overview** — The project components, VM layout, network mode, the 5-member work-allocation table, and a diagram or clear description of how the pieces connect.
- **Build steps** — The key configuration decisions — not a copy of every command, but enough that a reader could reproduce the system. Reference the actual config files you submit.
- **Security** — Firewall policy and the justification for each open port; SSH hardening choices and trade-offs; fail2ban and auditd setup; your Lynis score and the three findings you remediated.
- **Backup & restore** — What you back up, how, the schedule, the retention policy, and the exact restore procedure — with evidence you tested it.
- **Automation** — Each script: its purpose, parameters, error handling, and alerting. Include the shellcheck result.
- **TLS (if attempting the bonus)** — The §7.1 reasoning plus your configuration.
- **Reflection** — What broke, what you would improve, and what you learned. Honest reflection scores well.

Screenshots are evidence, not decoration

Every claim in the security, backup, and automation sections should be backed by a labelled screenshot or a real config file in the archive. "It works" without evidence is treated as not shown.

9. Deliverable 2 — Live Demo

A 12–15 minute live demonstration on your own VMs. This carries the single largest block of marks (the 30% "System works" criterion plus most of Backup and Automation), so plan and rehearse it.

Run of show (suggested)

1. Boot / show the running system and open the firewall + service status (1–2 min).
2. Show the whole system working end to end — all three components (2–3 min).
3. Security: trigger a fail2ban ban; show auditd catching access to a sensitive file (2 min).
4. Backup & restore: destroy live data, restore it from your automated backup, prove integrity (2–3 min).
5. Automation: cause a failure that your health-check detects and alerts on (1–2 min).
6. TLS bonus, if attempted (1 min).

Take a snapshot first

Snapshot a known-good VM state before you present. If a live step goes wrong, you can roll back and continue rather than losing the whole demo. A recovered demo scores; a frozen one does not.

10. Deliverable 3 — Oral Defence (Q&A)

After the demo the lecturer will ask 3–5 technical questions about your system. The goal is to confirm that you understand what you built and did not simply follow a tutorial. Be ready to explain any decision.

Representative questions:

- Why did you open exactly those ports, and what would an attacker gain from each?
- Walk me through what happens, step by step, when a request hits the web server (Nginx/Apache) and reaches your app/database.
- Your SSH is hardened how, and what specifically does each setting prevent?
- Show me where your secrets live and who can read that file. Why there?
- Your backup runs at 2 AM — what happens if the disk is full when it runs? Does your script notice?
- Restore this database in front of me from a backup taken earlier.
- What does `set -euo pipefail` change in this script, and where would it have saved you?
- (TLS) Why can't you use Let's Encrypt on this VM, and what did you do instead?

11. Grading Rubric

The project is worth 20% of the course grade, marked on a 10-point scale with up to +1 bonus. The five criteria below match the course syllabus. Within each, marks are awarded by the band your work falls into.

Weight	Criterion	Evidence
30%	System works	Live demo — all three components run end to end
20%	Security	UFW, SSH hardening, fail2ban, auditd, Lynis
20%	Backup & restore	Live restore recovering real data
15%	Automation	Scripts running, error handling, alerting, shellcheck
15%	Report & defence	Report quality + oral Q&A
+1	Bonus	Working self-signed TLS with correct reasoning (\$7)

Banding within each criterion

Band	What it looks like
Full marks	Works cleanly and live; correct, secure, and well-justified; student explains it confidently.
Most marks	Works with minor gaps or a small stumble; mostly correct; explanation solid.
Partial	Partially working or shown only by screenshot; noticeable gaps; explanation shaky.
Little / none	Not shown live, broken, copied without understanding, or cannot be explained.

11.1 Penalties

- A live secret (real password/key/token) in the submission: deduction under Security.
- A backup that cannot be restored during the demo: no Backup marks, however good the backup script looks.
- Scripts that fail shellcheck with errors, or ignore failures (no set -e / trap): reduced Automation marks.
- Cannot explain a component during defence: that component scores as "not understood" even if it runs.
- Late submission: per the program's standard late policy.

11.2 Academic integrity

You may use tutorials, documentation, and AI assistants to learn and build — that reflects real practice. But the system is yours and you must understand every part of it. The oral defence exists precisely to verify this: work you cannot explain earns no marks, and passing off another student's system as your own is treated as academic misconduct under Faculty rules.

12. Requirements Checklist

A single compliance matrix of every mandatory requirement, mapped to the grading criterion it feeds. Tick each box as you complete it. Every item is mandatory (the "Applies" column reads "All") because the project is one unified system. This is a build checklist — use it while working; §13 is the final pre-demo check.

✓	Requirement	Applies	Criterion
System & storage			
☐	Clean Ubuntu 22.04 or CentOS Stream 9 install; meaningful hostname set	All	System
☐	Non-root administrative (sudo) user used for daily work	All	Security
☐	Dedicated data/backup mount via /etc/fstab, surviving reboot	All	System
☐	2-VM (or more) architecture (mandatory for a team of 5)	All	System
Security hardening			
☐	Host firewall (UFW or firewalld) default-deny inbound; only needed ports open	All	Security
☐	SSH hardened: root login off, key auth, non-default port or AllowUsers	All	Security
☐	fail2ban protecting SSH; a ban can be triggered live	All	Security
☐	auditd watching /etc/passwd and /etc/shadow; trail readable	All	Security
☐	Lynis run recorded; score noted; 3 findings remediated	All	Security
Backup & restore			
☐	Automated backup: compressed, timestamped, with a stated retention policy	All	Backup
☐	Backup scheduled via cron or systemd timer; schedule documented	All	Backup
☐	Restore procedure rehearsed; recovers real data live	All	Backup
Automation			
☐	Health-check script inspects real state and reacts (log/alert)	All	Automation
☐	Scripts use set -euo pipefail and a trap on ERR/EXIT	All	Automation
☐	All scripts pass shellcheck (clean, or justified warnings noted)	All	Automation
☐	Working alert channel (mail / msmtp / Telegram) fires on simulated failure	All	Automation
Component 1 — Infrastructure, Web & Security (mandatory)			
☐	Web server (Nginx/Apache) reverse proxy in front of the app, correct header forwarding	All	System
☐	≥2 virtual hosts on the same web server (app site + at least one more)	All	System
☐	2-VM (or more) architecture with a connection diagram in the report	All	System
Component 2 — Application & Database (mandatory)			
☐	App (Flask/FastAPI/Node) with at least one read and one write endpoint	All	System
☐	DB (PostgreSQL or MySQL): least-privilege role; localhost only (ss)	All	Security

✓	Requirement	Applies	Criterion
☐	App runs as a systemd service: Restart=on-failure, EnvironmentFile, journald	All	System
☐	Secrets in EnvironmentFile with restricted permissions (not hard-coded)	All	Security
☐	Kill the app process → systemd auto-restarts it (seen in journalctl)	All	System
Component 3 – Operations & Automation (mandatory)			
☐	Interactive CLI menu dispatching to each tool; handles invalid input	All	Automation
☐	Deploy action with safe test → reload → confirm and rollback on failure	All	Automation
☐	Backup action: dump DB + web, compress, timestamp, retention, rsync to 2nd VM	All	Backup
☐	Restore from a toolkit-produced backup, recovering real data	All	Backup
☐	Health-check action: CPU/disk/RAM/ports/service vs thresholds, with alert	All	Automation
☐	Log-rotation action: rotate, compress, keep N, drop older	All	Automation
☐	Each tool: getopts where sensible, --help, clear logging; whole set passes shellcheck	All	Automation
Bonus – optional (+1 mark)			
☐	Self-signed TLS serving the site/app; HTTP→HTTPS redirect; 443 open	All	Bonus
☐	Report explains why Let's Encrypt cannot be used on an isolated VM	All	Bonus

13. Pre-Submission Checklist

Tick every box before you present. If one is unchecked, fix it first.

System & baseline

- ☐ Hostname set; non-root sudo user in use; root SSH login disabled.
- ☐ Dedicated data/backup mount persists across reboot (checked in /etc/fstab).
- ☐ Firewall (UFW or firewalld) default-deny inbound; only required ports open and each one justified.
- ☐ fail2ban protecting SSH; a ban can be triggered live.
- ☐ auditd watching /etc/passwd and /etc/shadow; trail is readable.
- ☐ Lynis run recorded; three findings remediated.

The three project components

- ☐ Web server (Nginx/Apache) reverse proxy + ≥ 2 virtual hosts responding (Component 1).
- ☐ App serving end to end through the web server; database on localhost only, verified with ss (Component 2).
- ☐ App runs as a systemd service and auto-restarts after a kill (Component 2).
- ☐ Menu dispatches to every tool; deploy has a working rollback (Component 3).

Backup & automation

- ☐ Backup is automated (cron/timer), compressed, timestamped, with retention.
- ☐ Restore rehearsed and recovers real data.
- ☐ Health-check script detects a real fault and fires an alert.
- ☐ All scripts use set -euo pipefail + trap and pass shellcheck.

Deliverables

- ☐ Report (PDF) complete, with evidence for every security/backup/automation claim.
- ☐ Archive built: report + scripts + real config files + README.md, no live secrets.
- ☐ VM snapshot taken of a known-good state before the demo.
- ☐ Demo rehearsed end to end within the time limit.
- ☐ (Bonus) Self-signed TLS works and the "why not Let's Encrypt" reasoning is in the report.